MULTITHREADED LINUX APPLICATION – POSIX THREADS, MUTEXES AND THE PRODUCER-CONSUMER PROBLEM

A Project Report Submitted in the partial fulfillment of the requirements of the course titled **"Operating Systems and Systems Programming (25CS2104E)"**

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

Name	Roll No.
M. BALA PRANITHA	2520030051
S. MANVI VAISHNAVI	2520030326
D. ROHINI TANMAYI	2520030349

Under the Esteemed Guidance of

Guide Name: [Faculty Name]

Designation: [Designation]

Department of Computer Science and Engineering

K L (Deemed to be) University (Deemed to be University estd. u/s. 3 of the UGC Act, 1956) Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Declaration

The Project Report entitled **"MULTITHREADED LINUX APPLICATION – POSIX THREADS, MUTEXES AND THE PRODUCER-CONSUMER PROBLEM"** is a record of bonafide work of **M. BALA PRANITHA – 2520030051, S. MANVI VAISHNAVI – 2520030326, D. ROHINI**

TANMAYI – 2520030349, submitted in partial fulfillment of the requirements of the course titled “Operating Systems and Systems Programming (25CS2104E)” under the B.Tech II Year Trimester - III program in the Department of Computer Science and Engineering at K L University. The results presented in this report have not been copied from any other department, university, or institute.

M. BALA PRANITHA – 2520030051 S. MANVI VAISHNAVI – 2520030326 D. ROHINI TANMAYI – 2520030349

CERTIFICATE

This is to certify that the project based report entitled **“MULTITHREADED LINUX APPLICATION – POSIX THREADS, MUTEXES AND THE PRODUCER-CONSUMER PROBLEM”** is a bonafide work done and submitted by **M. BALA PRANITHA – 2520030051, S. MANVI VAISHNAVI – 2520030326, D. ROHINI TANMAYI – 2520030349** in partial fulfillment of the requirements of the course titled “Operating Systems and Systems Programming (25CS2104E)” under the B.Tech II Year Trimester - III program in the Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year 2026-2027.

Signature of the Guide

Signature of the Course Coordinator Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our Course Coordinator **[Course Coordinator Name]** and our guide **[Guide Name]**, Department of Computer Science and Engineering, for their tremendous support, encouragement and guidance throughout this project.

We express our gratitude to **[HOD Name]**, Head of the Department, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

M. BALA PRANITHA – 2520030051 S. MANVI VAISHNAVI – 2520030326 D. ROHINI TANMAYI – 2520030349

ABSTRACT

The project titled “Multithreaded Linux Application – POSIX Threads and Mutexes Demo” demonstrates how a large computational task can be divided into smaller tasks and processed concurrently on a Linux system. In modern computing, a single flow of execution rarely makes full use of the available CPU cores, and naive sharing of data between several flows of execution leads to race conditions and incorrect results. This project addresses both issues using the classic Producer-Consumer design.

The application, written in C and executed on Ubuntu Linux, processes a dataset of 2,000,000 elements. A single producer thread divides the dataset into 200 chunks of 10,000 elements each and places them into a bounded buffer of 8 slots. Four worker (consumer) threads, created with the POSIX Threads (Pthreads) library, remove chunks from the buffer and compute partial results, which are finally combined into one answer. Access to the shared buffer is protected with a mutex so that only one thread modifies it at a time.

To evaluate the design, the program first runs a sequential baseline and then the multithreaded version, compares both results for correctness, and reports execution times and speedup. The project demonstrates core operating-system concepts: threads, synchronization, mutual exclusion, shared resources, bounded-buffer communication and parallel processing.

INDEX

S. No.	Chapters	Topics
–	Acknowledgement, Abstract	–

S. No.	Chapters	Topics
1	Introduction	1.1 Background of the project; 1.2 Problem statement
2	System Architecture	2.1 High-level architecture / block diagram; 2.2 Module and data-structure diagram
3	CO's Attainments	3.1 – 3.6 Course outcome attainment
4	Screen Shots	4.1 Source code and terminal output
5	Testing	5.1 Test cases and results
6	Future Enhancements	6.1 Planned features
7	Conclusion	7.1 Conclusion
8	References	Books, tutorials, documentation sites used
9	Appendices	Setup instructions, user manual, geo-tag photos, review forms

---

CHAPTER 1 – INTRODUCTION

1.1 Background of the Project

Early computers executed one instruction stream at a time, and a program was a single sequence of steps from start to finish. As hardware evolved, processors gained multiple cores, and operating systems such as Linux gained the ability to schedule several independent flows of execution at once. Today even an ordinary laptop exposes several CPU cores; the machine used in this project reported 8 detected cores. A program that does not use more than one flow of execution leaves most of that capacity idle.

Processes and threads. A process is a program in execution with its own address space. A thread is a smaller unit of execution inside a process. All threads of a process share the same memory, which makes communication between them fast, but also dangerous: if two threads change the same data at the same time, the result becomes unpredictable. This situation is called a **race condition**.

POSIX Threads (Pthreads). Pthreads is the standard C library interface for creating and managing threads on Linux and other POSIX systems. It provides functions to create threads, wait for them to finish, and synchronize them with mutexes and condition-based

waiting. Programs using it are compiled with the `-pthread` flag.

Mutual exclusion. A mutex (mutual exclusion lock) allows only one thread at a time into a critical section. A thread locks the mutex before touching shared data and unlocks it afterwards. A useful analogy is a room with a single key: only the person holding the key can enter.

The Producer-Consumer problem. This is one of the classical synchronization problems in operating systems. One or more producers generate work items and place them into a shared buffer; one or more consumers remove items and process them. Because the buffer has a fixed size (a bounded buffer), the producer must wait when it is full, and consumers must wait when it is empty. A restaurant is a good analogy: the chef (producer) places orders on a counter (buffer) with limited space, and waiters (consumers) pick them up.

Parallel processing and speedup. Dividing a large task among threads allows different parts to run simultaneously. The benefit is measured by speedup, defined as Sequential Time ÷ Parallel Time. Speedup is not automatic: creating threads, locking and waiting all cost time, so for small workloads the multithreaded version may not be faster. Measuring and understanding this is one of the learning goals of the project.

This project, **Multithreaded Linux Application using POSIX Threads, Mutexes and the Producer-Consumer Problem**, brings these ideas together in a single working program and verifies it by comparing it against a sequential baseline.

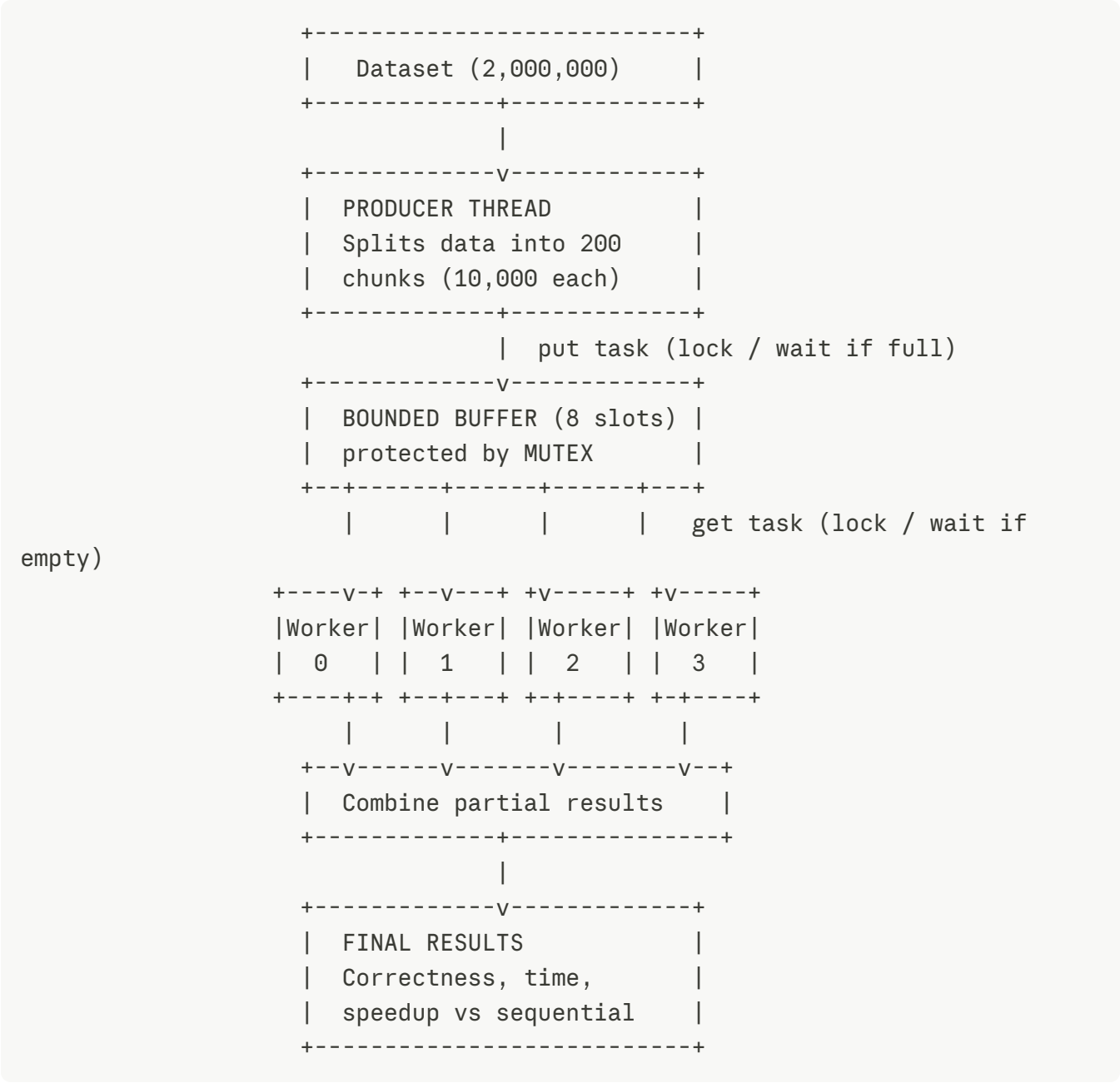
1.2 Problem Statement

A single-threaded program processes a large dataset one element after another and cannot exploit multiple CPU cores. A naive multithreaded program, on the other hand, lets several threads read and write shared data without coordination, causing race conditions, corrupted buffer state and incorrect results. There is also the question of how to hand work from the thread that creates it to the threads that process it without overflowing storage or letting workers take tasks that do not exist.

The core problem is to design and implement a multithreaded Linux application in C that (a) divides a dataset of 2,000,000 elements into smaller tasks, (b) transfers those tasks from a producer to multiple worker threads through a bounded buffer, (c) protects all shared buffer operations with proper synchronization, (d) combines the workers' partial results into one correct final result, and (e) compares the result and the execution time against a sequential baseline to demonstrate correctness and speedup.

CHAPTER 2 – SYSTEM ARCHITECTURE

2.1 High-level Architecture Diagram



The system has four logical layers:

Layer	Role
Work generation	The producer thread divides the 2,000,000-element dataset into 200 tasks of 10,000 elements and inserts them into the buffer.

Layer	Role
Communication	A bounded buffer with 8 slots holds tasks between producer and consumers. A mutex guards every insert and remove; the producer waits when the buffer is full and workers wait when it is empty.
Processing	Four worker threads repeatedly take a task, compute a partial result for that index range, and keep it for the final combination.
Reporting	After all tasks are processed, partial results are combined; the program prints the FINAL RESULTS section with correctness check, both execution times and the speedup.

The program configuration observed at run time was:

Parameter	Value
Detected CPU cores	8
Dataset size	2,000,000 elements
Chunk size	10,000 elements per work item
Number of chunks	2,000,000 ÷ 10,000 = 200
Bounded buffer size	8 slots
Worker threads	4

2.2 Module and Data-structure Diagram

The program is organised around the following components:

Component	Responsibility
`main()`	Detects CPU cores, prints configuration, runs the sequential baseline, then creates the producer and worker threads, joins them, and prints final results.

Component	Responsibility
Sequential routine	Processes all 2,000,000 elements in one flow of execution and records the time. Used as the baseline.
Producer routine	Creates the 200 tasks (start and end index of each chunk) and inserts them into the bounded buffer.
Worker (consumer) routine	Removes a task from the buffer, processes the range `[start, end)`, prints a line such as `[Worker 0] processed range [0, 10000)`, and adds to its partial result.
Bounded buffer	A fixed array of 8 task slots with in/out positions and a count, shared by all threads.
Synchronization objects	A mutex protecting the buffer, together with waiting logic for the “buffer full” and “buffer empty” conditions.
Result combiner and timer	Adds the partial results, compares with the sequential result, measures both times and computes speedup.

As the producer and the workers run at the same time, the workers do not process chunks in a fixed order: whichever worker is free takes the next available task.

CHAPTER 3 – CO’s ATTAINMENT

Note: the CO statements below should be replaced with the exact wording from the official OSSP (25CS2104E) syllabus before submission; the mapping of project concepts is what this report contributes.

3.1 CO1 Attainment

CO1 Syllabus	CO1 Concepts Included in Project
Understand the fundamental concepts of operating systems: processes, threads, scheduling and the role of the OS in managing CPU resources.	Process vs thread, CPU core detection, how Linux schedules threads across cores, sequential vs concurrent execution.

3.1.1 Scenario: The program detects the number of CPU cores (8) and runs the same workload in two ways, once with a single flow of execution and once with four worker threads, showing the difference between hardware resources (cores) and software units of work (threads).

3.2 CO2 Attainment

CO2 Syllabus	CO2 Concepts Included in Project
Create and manage multiple threads using system-level programming interfaces.	POSIX Threads: `pthread_create`, `pthread_join`, thread functions, passing arguments, `-pthread` compilation.

3.2.1 Scenario: One producer thread and four worker threads are created with Pthreads and joined before the final results are computed.

3.3 CO3 Attainment

CO3 Syllabus	CO3 Concepts Included in Project
Apply synchronization mechanisms to protect shared resources and avoid race conditions.	Mutex locking and unlocking, critical section, race condition prevention, mutual exclusion.

3.3.1 Scenario: Every insert into and removal from the shared bounded buffer happens only while the mutex is held, so two threads can never modify the buffer state simultaneously.

3.4 CO4 Attainment

CO4 Syllabus	CO4 Concepts Included in Project
Solve classical inter-thread communication problems such as Producer-Consumer.	Bounded buffer, producer waits when full, consumers wait when empty, work distribution.

3.4.1 Scenario: The producer fills 8 slots and must wait until a worker frees one; workers must wait when the buffer is empty. This avoids both overflow and the removal of non-existent tasks.

3.5 CO5 Attainment

CO5 Syllabus	CO5 Concepts Included in Project
Develop systems programs in C on Linux using the compiler toolchain and terminal.	`nano`, `gcc producer.c -o producer -pthread`, `./producer`, Linux terminal workflow, Ubuntu environment.

3.5.1 Scenario: The program is written in the Nano editor, compiled with `gcc` and run from the Ubuntu terminal.

3.6 CO6 Attainment

CO6 Syllabus	CO6 Concepts Included in Project
Evaluate performance and correctness of concurrent programs.	Sequential baseline, correctness check, execution timing, speedup = Sequential time ÷ Parallel time, overhead of thread creation and synchronization.

3.6.1 Scenario: The final results compare the sequential and multithreaded results for equality and report both times and the speedup, showing that more threads do not automatically mean faster execution.

CHAPTER 4 – SCREEN SHOTS

4.1 Screen Shots

Screenshot 1 – Commands used (editor, compile, run)

```
nano producer.c
gcc producer.c -o producer -pthread
./producer
```

Command	Meaning
`nano producer.c`	Opens the Nano editor to create or edit the C source file.
`gcc producer.c -o producer -pthread`	Compiles the source with GCC, names the executable `producer`, and enables POSIX thread support.
`./producer`	Runs the compiled program.

[Insert screenshot of the terminal here]

Screenshot 2 – Program configuration output

```
Detected CPU cores  : 8
Dataset size        : 2000000 elements
Chunk size          : 10000 elements/work item
Bounded buffer size : 8 slots
Worker threads      : 4
```

[Insert screenshot here]

Screenshot 3 – Sequential baseline

```
>>> Running sequential baseline...
Sequential result = 95998839, time = 0.0071 s
```

[Insert screenshot here]

Screenshot 4 – Multithreaded producer-consumer execution

```
>>> Running multithreaded producer-consumer version
[Worker 0] processed range [0, 10000)
[Worker 1] processed range [10000, 20000)
[Worker 2] processed range [20000, 30000)
[Worker 3] processed range [30000, 40000)
...
```

[Insert screenshot here]

Screenshot 5 – FINAL RESULTS section (sequential result, multithreaded result, correctness check, both execution times and speedup)

[Insert screenshot of the FINAL RESULTS section here]

Screenshot 6 – Source code (`producer.c`)

[Insert screenshots of the source code here, section by section: headers and shared structures, producer routine, worker routine, buffer put/get with mutex, main().]

CHAPTER 5 – TESTING

5.1 Test Cases and Results

Note: the observed values for sequential result and time come from the actual run. Fill the remaining "Observed" entries from your own runs and attach screenshots.

TEST CASE 1: Standard run (TST-OSSP-001)

Objective: Verify that the program divides the dataset correctly and finishes with the default configuration.

Input Parameter	Value	Expected Result
Dataset size	2,000,000	–
Chunk size	10,000	200 tasks created
Buffer size	8 slots	At most 8 tasks waiting at once
Worker threads	4	Workers 0–3 all process tasks

TEST CASE 2: Correctness check (TST-OSSP-002)

Objective: Verify that the multithreaded result equals the sequential result.

Item	Value
Sequential result	95998839 (observed)
Multithreaded result	Must be 95998839
Correctness check	Must report a match

TEST CASE 3: Range coverage (TST-OSSP-003)

Objective: Verify that every element is processed exactly once.

Check	Expected
Ranges printed by workers	`[0, 10000)` through `[1990000, 2000000)` with no gaps and no overlaps
Total ranges processed	200

TEST CASE 4: Bounded buffer behaviour (TST-OSSP-004)

Objective: Verify blocking behaviour when the buffer is full or empty.

Situation	Expected behaviour
Buffer full (8 tasks)	Producer waits until a worker removes a task
Buffer empty	Workers wait until the producer adds a task

TEST CASE 5: Repeated runs (TST-OSSP-005)

Objective: Verify that the result is stable across several runs even though the order of worker output changes.

Check	Expected
Final result over 5 runs	Identical every time
Worker output order	May differ between runs

TEST CASE 6: Performance comparison (TST-OSSP-006)

Objective: Measure speedup.

Metric	Value
Sequential time	0.0071 s (observed)
Multithreaded time	[fill from your run]
Speedup	Sequential ÷ Multithreaded = [fill]

Observation: With a small, simple per-element computation, the cost of creating threads, locking and waiting can be comparable to or larger than the useful work, so the multithreaded version is not guaranteed to be faster than the sequential one.

CHAPTER 6 – FUTURE ENHANCEMENTS

6.1 Planned Features

The current application is a compact but complete demonstration of Pthreads, mutexes and the Producer-Consumer pattern. Several improvements are possible:

- Multiple producer threads feeding the same bounded buffer.
- A configurable number of workers, chunk size and buffer size from the command line.
- Reading real input data from a file instead of a generated dataset.
- Heavier per-chunk computation (sorting, image or matrix processing) to show clearer speedup.
- Using a lock-free queue or atomic operations to reduce locking overhead.
- A thread-pool design that reuses workers for several jobs.
- Detecting deadlock and starvation, and adding logging of thread timelines.
- A graphical or web dashboard showing buffer occupancy and per-worker load.
- A comparison with other approaches such as OpenMP or multiple processes with shared memory.

CHAPTER 7 – CONCLUSION

The Multithreaded Linux Application is a successfully developed C program that demonstrates how a large workload can be divided among multiple threads on Linux while still producing a correct result. Using POSIX Threads, one producer divides a 2,000,000-element dataset into 200 tasks, a bounded buffer of 8 slots carries them, and four worker threads process them concurrently. A mutex protects all shared buffer operations and prevents race conditions, and the producer and consumers wait correctly when the buffer is full or empty.

The sequential baseline and the multithreaded version produce the same final result, and the timing comparison shows that speedup depends on the size of the workload and the overhead of synchronization. The project strengthened our understanding of processes and threads, mutual exclusion, the Producer-Consumer problem, shared resources and parallel processing, and gave practical experience of compiling and running systems programs in Ubuntu Linux. It forms a foundation for more advanced work in concurrent and parallel programming.

CHAPTER 8 – REFERENCES

Books

- Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*, Wiley.
- Tanenbaum, A. S., & Bos, H. *Modern Operating Systems*, Pearson.
- Stevens, W. R., & Rago, S. A. *Advanced Programming in the UNIX Environment*, Addison-Wesley.
- Kerrisk, M. *The Linux Programming Interface*, No Starch Press.
- Kernighan, B. W., & Ritchie, D. M. *The C Programming Language*, Prentice Hall.

Tutorials & Learning Resources

- GeeksforGeeks – Producer-Consumer problem and multithreading in C: www.geeksforgeeks.org
- Unwired Learning – Producer-Consumer with semaphores: unwiredlearning.com
- Linux manual pages for `pthread_create`, `pthread_join`, `pthread_mutex_lock`: man7.org
- GNU C Library documentation: www.gnu.org/software/libc

These sources were used to understand threads, synchronization and the Producer-Consumer problem.

CHAPTER 9 – APPENDICES

A. Source Code Files

File	Description
`producer.c`	Complete program: sequential baseline, producer, workers, bounded buffer with mutex, result combination and timing.

B. Installation / Setup Instructions

1. Use Ubuntu Linux (native, virtual machine or WSL).
2. Install the compiler if needed: `sudo apt update && sudo apt install build-essential`.
3. Create or open the source file: `nano producer.c`.
4. Compile: `gcc producer.c -o producer -pthread`.
5. Run: `./producer`.

C. User Manual

The program needs no input. On running it prints the configuration, executes the sequential baseline, then the multithreaded producer-consumer version, and finally prints the FINAL RESULTS section.

D. Key Terms

Term	Easy explanation
Process	A program that is currently running
Thread	A smaller execution unit inside a process
Multithreading	Running multiple threads within one program

Term	Easy explanation
Pthreads	A C library for creating and managing threads
Mutex	A lock that protects shared resources
Race condition	A fault caused by threads interfering with each other
Producer / Consumer	Creates tasks / takes and processes tasks
Bounded buffer	Temporary storage with a fixed capacity
Synchronization	Coordinating threads so they work correctly
Speedup	Sequential time ÷ parallel time

E. Software Requirements

- Programming language: C (GCC)
- Operating system: Ubuntu Linux
- Editor: Nano / VS Code
- Library: POSIX Threads (`-pthread`)

F. Geo Tag Photos with Guide

[Insert geo-tagged photos with the guide here]

G. Review Forms with Guide Signatures

[Insert signed review forms here]